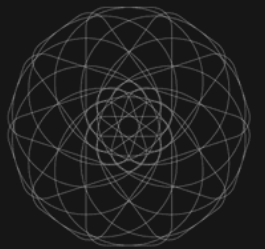




•• Matplotlib & Seaborn Guía



2025/08/04



CONTENIDO

01

Matplotlib Base

02

Seaborn Estadístico

03

Combinación Práctica

Matplotlib Base



Matplotlib: Control Total

La base para un control granular sobre cada elemento de la figura, ofreciendo máxima flexibilidad para personalizar gráficos científicos o para publicaciones.

Creación de Gráficos Básicos



Líneas y Puntos

```
plt.plot(x, y)
```



Dispersión

```
plt.scatter(x, y)
```



Histograma

```
plt.hist(data)
```

Estas funciones elementales son los bloques para construir visualizaciones complejas y reproducibles.

Personalización y Etiquetado

Un texto claro y posicionado correctamente reduce la interpretación errónea y permite que la audiencia capture el mensaje central.

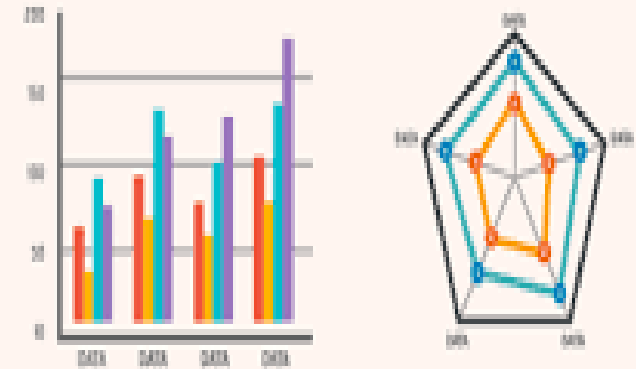
```
plt.title('Título descriptivo')
```

```
plt.xlabel('Etiqueta del Eje X')
```

```
plt.ylabel('Etiqueta del Eje Y')
```

```
plt.legend()
```

matplotlib



Gestión de Figuras

Figura Simple

Crea un lienzo nuevo con un tamaño específico.

```
plt.figure(figsize=(w, h))
```



Subgráficos

Crea múltiples gráficos en una misma figura.

```
plt.subplots(filas, cols)
```

Controlar la resolución y distribución de paneles facilita la comparación visual y la publicación.



Mostrar

```
plt.show()
```

Renderiza la figura en pantalla.



Guardar

```
plt.savefig('nombre.png')
```

Exporta la figura en alta calidad.

Aplicar Estilos Predefinidos

Cambiar de estilo mejora la legibilidad y adapta la estética a normas de publicación con una sola línea de código.

```
plt.style.use('nombre_estilo')
```

Ejemplos: 'ggplot', 'seaborn-v0_8'

Estilo 1

Apariencia clásica

Estilo 2

Apariencia moderna

Estilo 3

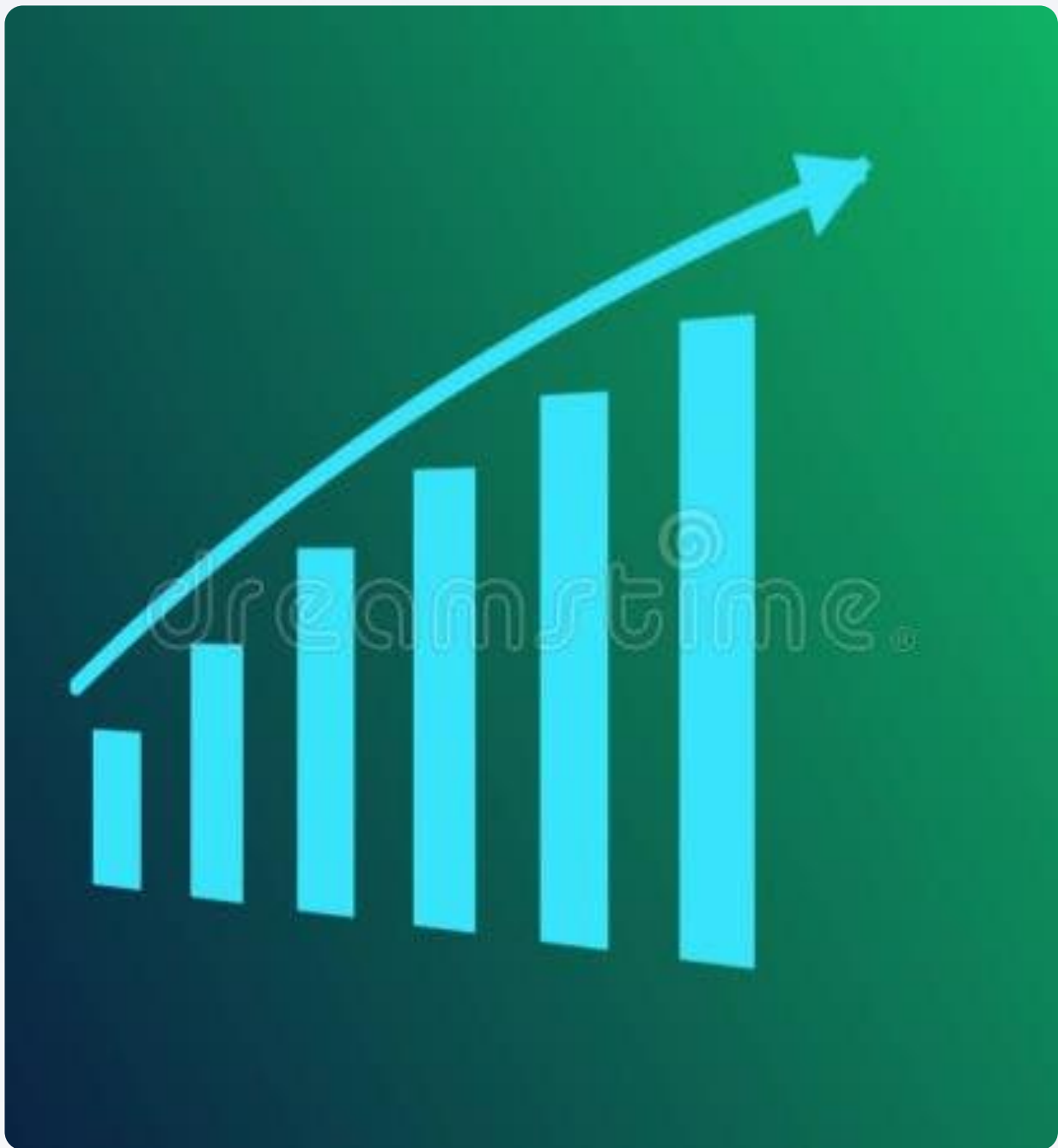
Apariencia publicación

Estilo 4

Apariencia oscura

Seaborn Estadístico





Seaborn: Estadísticas Estéticas

Construido sobre Matplotlib, simplifica la creación de gráficos estadísticos con menos código y mejor estética por defecto.

- Acepta DataFrames y nombres de columnas.
- Añade modelos de tendencia automáticamente.
- Permite centrarse en la interpretación , no en la configuración.

Visualizando Relaciones entre Variables



Scatter Plot

```
sns.scatterplot(data=df, x='col_x', y='col_y')
```



Line Plot

```
sns.lineplot(data=df, x='col_x', y='col_y')
```



Relational Plot

```
sns.relplot(..., col='categ')
```

Estas funciones integran intervalos de confianza y ajustes, facilitando la exploración de correlaciones.

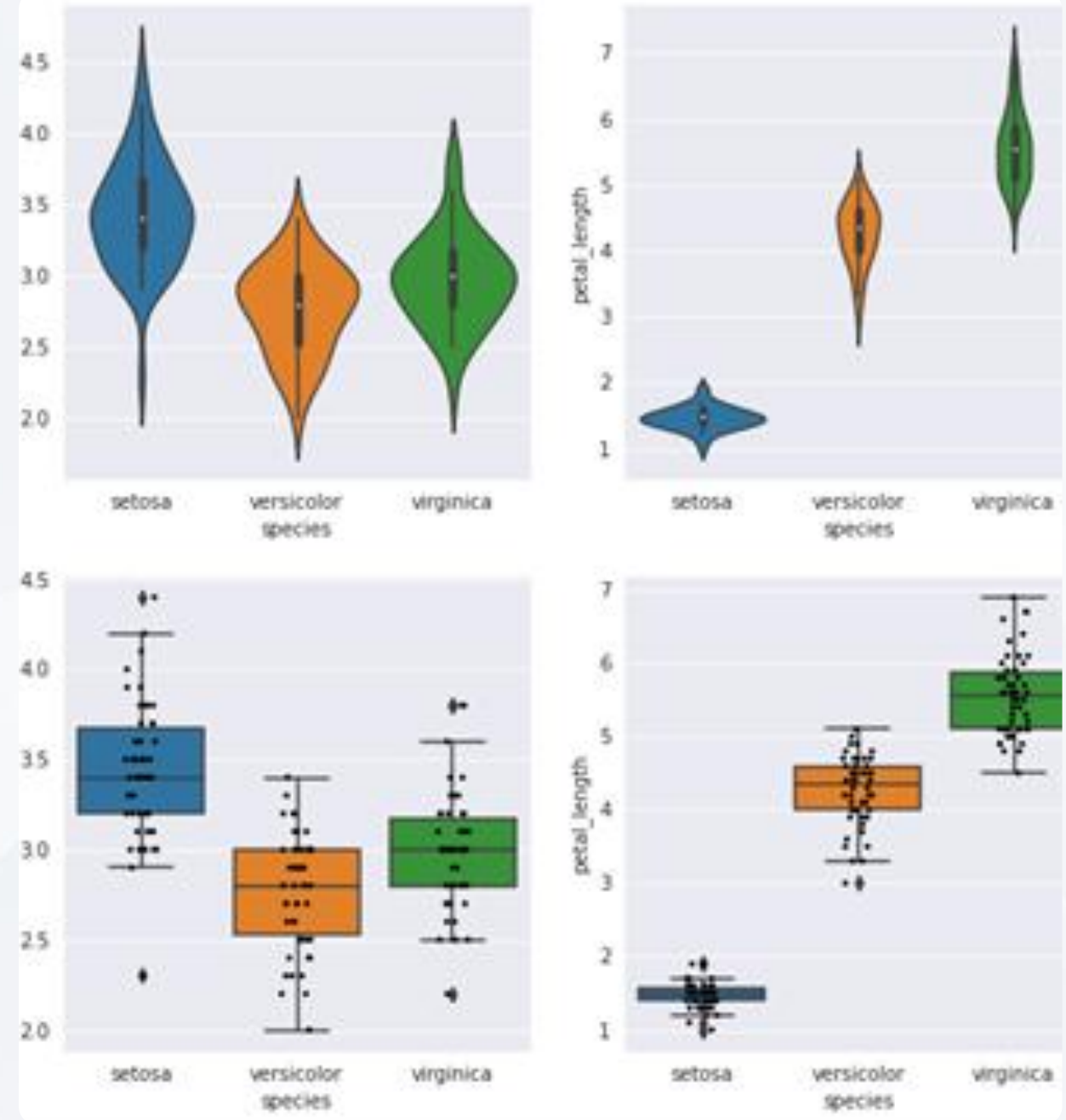
Distribuciones y Datos Categóricos

Gráficos que proporcionan una visión rica de la forma de los datos, permitiendo detectar sesgos y valores extremos.

`sns.histplot` / `kdeplot`: Estiman la distribución de una variable.

`sns.boxplot` / `violinplot`: Comparan estadísticas por categorías.

`sns.barplot`: Estiman una estadística (ej. media) por categoría.



Análisis Multivariado Integrado



Heatmap

Visualiza matrices de correlación o cualquier tabla de datos en un formato de mapa de calor, revelando patrones a un vistazo.

```
sns.heatmap(data)
```



Pair Plot

Crea una matriz de gráficos de dispersión para explorar rápidamente dependencias entre múltiples variables.

```
sns.pairplot(df)
```

Combinación Práctica





Uso Mixto: El Mejor de Dos Mundos

Seaborn crea el gráfico estadístico con rapidez y elegancia, mientras que Matplotlib se utiliza para ajustes finales de personalización.

Flujo de Trabajo Combinado

Importar ambas librerías permite aprovechar la velocidad de Seaborn y el control de Matplotlib, logrando visualizaciones totalmente personalizables sin sacrificar flexibilidad.

Ejemplo de Código Integrado

Un flujo típico: exploración rápida, estética automatizada y retoques finales manuales.

```
import matplotlib.pyplot as plt
import seaborn as sns
# Cargar dataset de ejemplo
tips = sns.load_dataset("tips")
# 1. Crear gráfico con Seaborn
sns.barplot(x="day", y="total_bill", data=tips, color="#00BCD4")
# 2. Personalizar con Matplotlib plt.title("Gasto Total Promedio por Día") plt.ylabel("Gasto Total ($)") #
3. Mostrar figura plt.show()
```

THANK YOU



Kimi AI
2025/08/04